

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
INGENIERÍA EN CIENCIAS Y SISTEMAS
ORGANIZACIÓN DE LENGUAJES Y COMPILADORES 2
SEGUNDO SEMESTRE 2026
PROYECTO 1



Andy Jefferson González Fuentes 202231338

MANUAL TÉCNICO

1. Introducción y alcance

Este manual técnico documenta la arquitectura interna, las decisiones de diseño y la implementación del compilador desarrollado para el Proyecto 1 del curso de Compiladores 2. Está dirigido a cualquier desarrollador que necesite mantener, extender o evaluar el código fuente del proyecto.

El sistema implementa el frontend y el backend completos (análisis léxico, sintáctico y semántico, generación de código intermedio de tres direcciones -C3D- y traducción a código C) para tres lenguajes: PigLatin, Y y Zetariano (Z); además de un IDE en JavaFX que integra todo el pipeline.

A diferencia del manual de usuario, este documento no explica cómo usar los menús de la aplicación, sino cómo está construida internamente: paquetes, clases, flujo de datos entre fases del compilador y las convenciones que debe seguir quien agregue código nuevo.

2. Tecnologías y dependencias

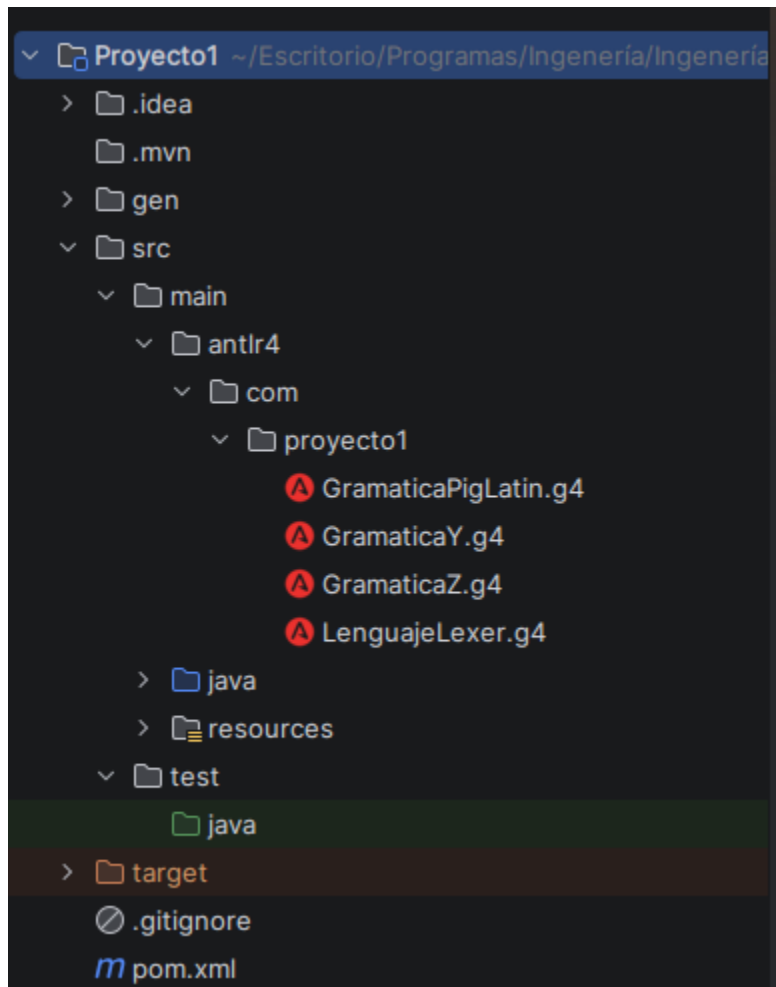
Componente	Versión / detalle
Lenguaje	Java 17 (maven.compiler.release=17)
Gestor de build	Apache Maven
Generador de parsers	ANTLR 4.13.1 (antlr4-maven-plugin, con visitor=true)
Interfaz gráfica	JavaFX 21.0.4 (javafx-controls, javafx-fxml)
Empaquetado / ejecución	javafx-maven-plugin 0.0.8, mainClass = com.proyecto1.ui.MainApp
Backend de generación	C (ANSI C), compilable con cualquier compilador estándar (gcc, clang)

El plugin de ANTLR compila las gramáticas ubicadas en src/main/antlr4 y coloca las clases generadas (lexer, parsers y contextos) en target/generated-sources/antlr4, por lo que no se versionan a mano.

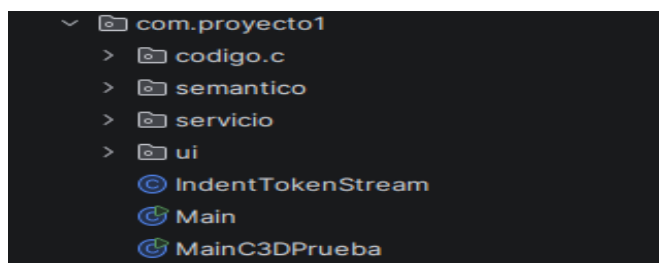
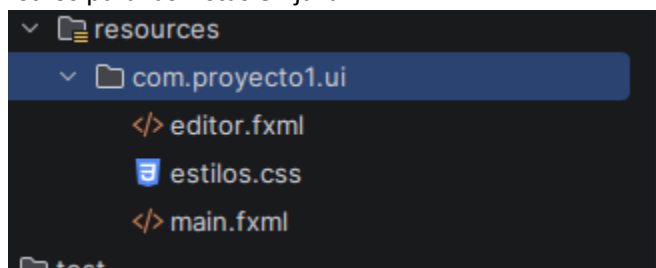
Comando de build y ejecución: "mvn javafx:run" desde la carpeta Proyecto1 (contiene el pom.xml).

3. Estructura del repositorio

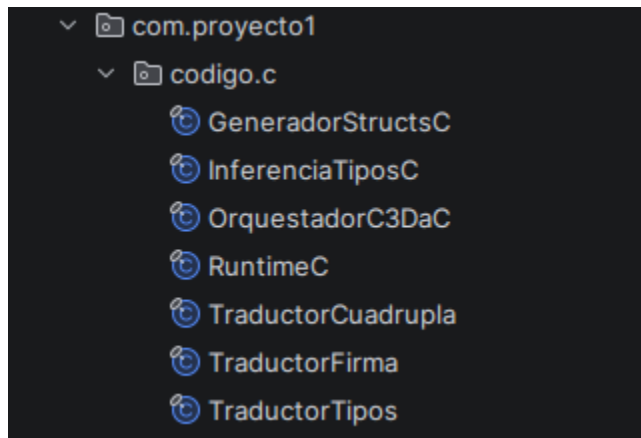
El repositorio contiene un único módulo Maven, Proyecto1, con la siguiente organización:



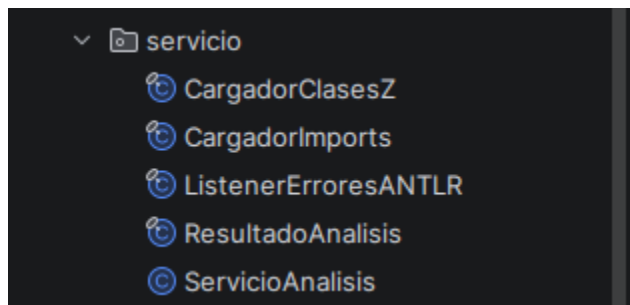
Estilos para las vistas en java fx



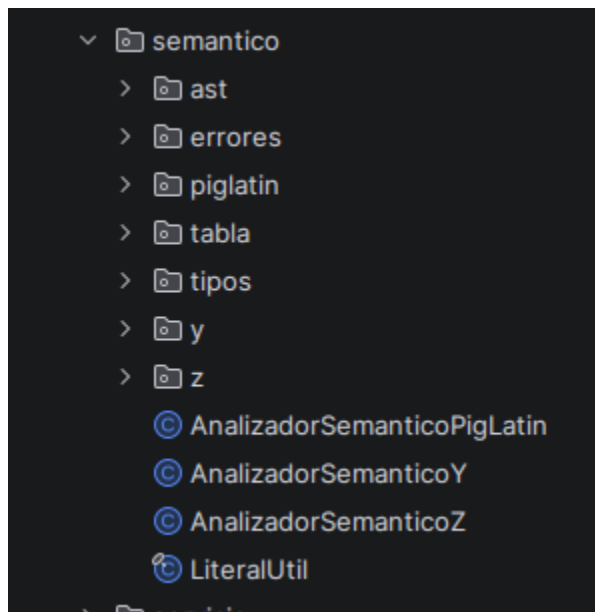
Traducir el código de tres direcciones a c

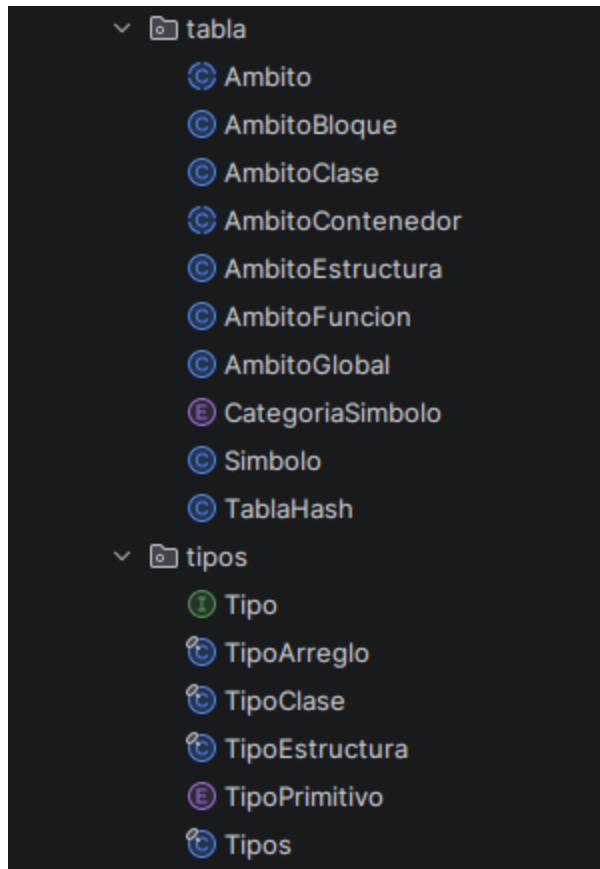


Conexión a la vista



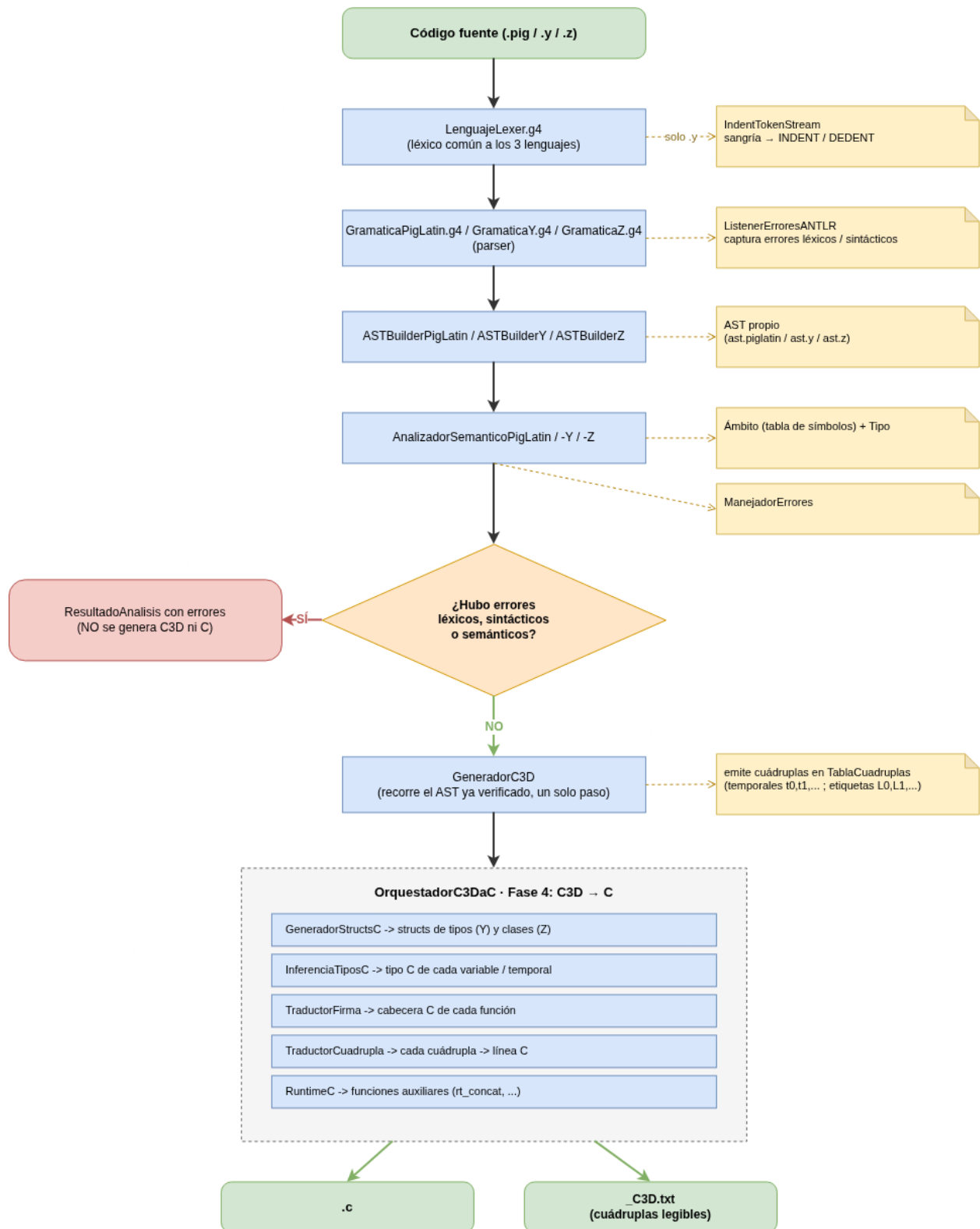
Análisis semántico





4. Pipeline de compilación

Cada archivo (.pig, .y o .z) se procesa a través de una única clase orquestadora, ServicioAnálisis, que encadena las fases del compilador siguiendo un patrón Strategy: un método privado por lenguaje (analizarY, analizarZ, analizarPigLatin), todos con la misma forma. El flujo completo es el siguiente:



5. Análisis léxico y sintáctico (ANTLR4)

El proyecto usa un único lexer compartido, `LenguajeLexer.g4`, y tres gramáticas de parser independientes (`GramaticaPigLatin.g4`, `GramaticaY.g4`, `GramaticaZ.g4`), todas con `tokenVocab = LenguajeLexer`. Cada gramática de parser define su propia regla de entrada (programa para PigLatin y Y, `CompilationUnit` para Z) y usa etiquetas de alternativa (`#nombreDef`) para que ANTLR genere un contexto y un método de visitor por cada producción, lo cual simplifica el `ASTBuilder` correspondiente.

5.1 Manejo de la indentación en Y (`IndentTokenStream`)

El lenguaje Y usa bloques delimitados por sangría, al estilo Python, en vez de llaves. Como ANTLR no soporta indentación de forma nativa, `LenguajeLexer.g4` emite un token `NEWLINE` por cada salto de línea real (dejando espacios/tabs/comentarios en un canal oculto), y la clase `IndentTokenStream` hace una segunda pasada sobre todo el flujo de tokens ya generado: agrupa los `NEWLINE` consecutivos, mide el nivel de sangría de la línea siguiente y sintetiza tokens `INDENT` o `DEDENT` según aumente o disminuya respecto al nivel anterior. Solo la gramática de Y usa este filtro; PigLatin y Zetariano usan directamente un `CommonTokenStream` de ANTLR porque delimitan sus bloques con llaves.

5.2 Reporte de errores (`ListenerErroresANTLR`)

`ListenerErroresANTLR` reemplaza el listener por defecto de ANTLR (que imprime a `stderr` en inglés) por dos listas propias en español. Una sola instancia se registra tanto en el lexer como en el parser: se autoclasifica revisando si el "recognizer" que reporta el error es un Lexer (error léxico) o no (error sintáctico). La columna se guarda tal como la entrega ANTLR (`getCharPositionInLine()`, base 0).

6. Árbol de sintaxis abstracta (AST)

Cada lenguaje tiene su propio paquete de nodos AST (`semantico.ast.piglatin`, `semantico.ast.y`, `semantico.ast.z`), contruidos por su respectivo `ASTBuilder` a partir del árbol de parseo de ANTLR (patrón visitor). Los nodos son independientes entre lenguajes: no hay una jerarquía común de "nodo AST genérico" más allá de exponer el método `generarC3D(GeneradorC3D)`, que cada nodo implementa para emitir sus propias cuádruplas y las de sus hijos, sin guardar estado propio (todo el estado de generación vive en el `GeneradorC3D` recibido como parámetro).

Este diseño (un `ASTBuilder` + un paquete de nodos por lenguaje, en vez de un único AST genérico) evita condicionales por lenguaje dispersos en el resto del compilador: el analizador semántico y el generador de C3D de cada lenguaje solo conocen los nodos de su propio paquete.

7. Tabla de símbolos (paquete `semantico.tabla`)

La tabla de símbolos se implementa como una cadena de Ámbitos (patrón similar a una lista enlazada de tablas hash): para resolver un nombre se busca primero en el ámbito actual y, si no aparece, se sube al ámbito padre, hasta llegar al `AmbitoGlobal` (el único sin padre). Cada Ámbito usa una `TablaHash` propia (implementación manual con encadenamiento, sin depender de `java.util.HashMap`) para sus símbolos locales.

8. Sistema de tipos (paquete `semantico.tipos`)

La interfaz Tipo define el contrato mínimo (nombre() legible para mensajes de error, y métodos por defecto esNumerico(), esArreglo(), esCompuesto(), esVoid()). Las implementaciones concretas son TipoPrimitivo (enum: entero, flotante, caracter, cadena, bool, void), TipoArreglo, TipoEstructura (tipos definidos por el usuario en Y) y TipoClase (clases de Zetariano).

La clase utilitaria Tipos centraliza las reglas de compatibilidad, en particular esAsignable(destino, origen), usada al validar declaraciones con inicializador, asignaciones, argumentos de llamada y sentencias de retorno. Permite promoción implícita de entero a flotante (no al revés) y trata un tipo desconocido (por un error previo) como compatible con cualquier cosa, para no generar una cascada de errores derivados de uno solo.

9. Manejo de errores semánticos

ManejadorErrores es un recolector simple (una lista de ErrorSemantico) que se instancia una vez por análisis y se pasa explícitamente (no por estático/singleton) a cada verificación en todo el árbol. Esta decisión permite analizar varios archivos en paralelo o en pruebas unitarias sin que se mezclen los errores de uno con los de otro. Cada nodo del AST reporta sus propios errores en el momento en que los detecta y continúa analizando a sus hijos igual, en vez de detenerse en el primer error, de modo que el usuario vea de una vez todos los problemas de su programa.

ErrorSemantico es una clase inmutable con línea, columna y mensaje; se reutiliza tal cual para representar errores léxicos, sintácticos y semánticos (lo que distingue a un error léxico de uno semántico es en qué lista de ResultadoAnálisis cae, no la forma de la clase).

10. Generación de código de tres direcciones (C3D)

GeneradorC3D concentra los servicios que necesita cualquier nodo del AST al traducirse a C3D: pedir un temporal nuevo (t0, t1, ...), pedir una etiqueta nueva (L0, L1, ... para saltos de si/mientras/para), emitir cuádruplas estructuradas, consultar el Ámbito para resolver tipos, construir etiquetas de función/método/constructor (mangling) y registrar la Firma de cada función emitida. Cada nodo recibe este generador como parámetro (programa.generarC3D(generador)); ningún nodo guarda estado propio de generación.

10.1 Cuádruplas y backpatching

TablaCuádruplas es la lista global y ordenada de cuádruplas: la única fuente de verdad del código generado. Un índice (siguienteIndice()) se mantiene estable durante toda la generación, lo que permite backpatching (reemplazar una cuádrupla ya emitida, típicamente un salto cuyo destino aún no se conocía, una vez que ese destino se resuelve). Cuádrupla es una interfaz sellada con una clase concreta por tipo de instrucción, entre ellas:

Cuádrupla	Uso
CuádruplaBeginFunc / EndFunc	Delimitan el cuerpo de una función/método/constructor
CuádruplaAsignacion	Asignación simple a una variable o temporal
CuádruplaBinaria / Unaria	Operaciones aritméticas, relacionales y lógicas

Cuádrupla	Uso
CuadruplaIfTrue / IfFalse / Goto / Etiqueta / Salto	Control de flujo (si, mientras, para, elegir)
CuadruplaCall / Param / Return	Llamadas a función/método, paso de argumentos y retorno
CuadruplaNew / NewArray	Reserva de memoria para objetos y arreglos (equivalente a malloc)
CuadruplaCampoCarga / CampoGuarda	Lectura/escritura de un campo o atributo (objeto.campo)
CuadruplaIndiceCarga / IndiceGuarda	Lectura/escritura de una posición de arreglo
CuadruplaPrint / Read	Entrada/salida (imprimir/print/println, leer/readln)

10.2 Mangling de nombres

Para evitar colisiones de nombres en el C generado, cada función/método/constructor recibe una etiqueta única:

- Función suelta de Y: la etiqueta es el nombre tal cual (sin el sufijo interno "#aridad" que usa la tabla de símbolos).
- Método de Zetariano: "Clase_metodo" (por ejemplo, Pila_apilar).
- Constructor de Zetariano: "Clase_init_aN", donde N es la aridad (cantidad de parámetros), construida por `GeneradorC3D.etiquetaConstructor(clase, aridad)`.
- Métodos y constructores reciben siempre "this" como primer parámetro implícito de su Firma; las funciones de Y no.

11. De C3D a C (paquete codigo.c)

OrquestadorC3DaC ensambla el archivo .c completo a partir de las cuádruplas y las Firma de un programa. Sus responsabilidades, delegadas cada una a una clase colaboradora, son:

Clase	Responsabilidad
GeneradorStructsC	Emite un typedef struct por cada ESTRUCTURA (Y) o CLASE (Z), en el orden de declaración.
InferenciaTiposC	Infiere el tipo C de cada variable local y temporal de una función a partir de sus cuádruplas y su Firma.

Clase	Responsabilidad
TraductorFirma	Traduce una Firma a la cabecera C de una función (sin llave ni ';', para reusarla como definición o como prototipo).
TraductorCuadrupla	Traduce una cuádrupla a su línea C equivalente, usando el mapa lugar→tipoC de InferenciaTiposC.
TraductorTipos	Traduce un Tipo del lenguaje a su representación en C (las clases y estructuras son siempre punteros, p. ej. Persona*, porque siempre viven en heap).
RuntimeC	Runtime C mínimo inyectado al inicio del archivo generado: rt_read_string, rt_concat, rt_strcmp, rt_int_to_string, rt_double_to_string, entre otras.

Esta separación permite que TraductorCuadrupla decida, por ejemplo, usar rt_concat cuando un '+' involucra un char*, o rt_strcmp cuando '==' / '!=' compara dos char* (en C puro estos operadores no hacen eso), sin mezclar esa lógica con la construcción de la cabecera de la función o de los typedef struct.

12. Programas de múltiples archivos

PigLatin (.pig) actúa como punto de entrada del proyecto: es el único de los tres lenguajes que genera una función "main" en el C3D. Un .pig puede importar funciones definidas en archivos .y y clases definidas en archivos .z del mismo proyecto; esa resolución la realiza CargadorImports, que analiza recursivamente los archivos importados (usando la misma ServicioAnálisis) y construye un AmbitoGlobal con sus símbolos disponibles para el .pig.

De forma análoga, una clase Z puede referenciar a otra clase Z "hermana" dentro del mismo proyecto (por ejemplo, una clase Pila que usa una clase Nodo). CargadorClasesZ localiza y carga esas clases hermanas a partir de la carpeta raíz del proyecto.

Cuando se generan las firmas de funciones/métodos/constructores importados o hermanos (para escribirlas como prototipos en el .c y evitar errores de "implicit declaration" al compilar con gcc), ServicioAnálisis.recolectarFirmasImportadas construye un mapa etiqueta→Firma recorriendo el Ámbito correspondiente, aplicando las mismas reglas de mangling de la sección 10.2.

12.1 C3D consolidado del proyecto

Cuando el análisis arranca desde un .pig, ServicioAnálisis reinicia un acumulador (ThreadLocal<StringBuilder>, para no mezclar análisis concurrentes) y va agregando las cuádruplas de cada archivo analizado (el .pig y todos los .y/.z que importa). Al terminar, vuelca todo el acumulado a un único archivo <nombrePig>_C3D.txt junto al fuente. Si se analiza un .y o .z suelto (sin pasar por un .pig), no se genera este consolidado.

13. Orquestación del análisis (paquete servicio)

ServicioAnálisis.analizar(archivo, texto) es el único punto de entrada al pipeline completo para un archivo. Determina el lenguaje por la extensión, delega en el método privado correspondiente (analizarY / analizarZ / analizarPigLatin) y captura cualquier excepción no controlada como un "error interno", de modo que un fallo inesperado en cualquier fase no haga caer el IDE.

ResultadoAnálisis es un DTO puro (no conoce JavaFX ni la consola): agrupa las tres listas de errores por separado (léxicos, sintácticos, semánticos), la lista de advertencias, un mensaje de resumen y, si el análisis fue exitoso, el GeneradorC3D resultante. Quien lo consuma (en este proyecto, MainController) decide el formato exacto de presentación (colores, prefijos "[OK]"/"[ERROR]", etc.).

14. Interfaz gráfica (paquete ui)

La interfaz está construida con JavaFX y FXML. MainApp carga main.fxml y aplica la hoja de estilos estilos.css; MainController es el controlador de la ventana principal y coordina el árbol de trabajo (ArbolController), las pestañas de edición (EditorController, una instancia por archivo abierto), la consola, la tabla de errores y la barra de estado.

La acción "Analizar" ejecuta ServicioAnálisis en un hilo de fondo (javafx.concurrent.Task) para no bloquear la interfaz mientras se procesa el archivo, y vuelca el ResultadoAnálisis en el hilo de JavaFX una vez terminado (setOnSucceeded).

14.1 Resaltado de sintaxis

ResaltadorSintaxis escucha los cambios de texto del editor y delega en HiloResaltado, una tarea en segundo plano que ejecuta únicamente el lexer (com.proyecto1.LenguajeLexer) sobre el texto completo -sin invocar al parser ni al análisis semántico- y produce una lista de fragmentos coloreados (TramoColoreado) según el tipo de token, usando MapaColores para asignar el color de cada categoría léxica. Al ejecutarse en un hilo aparte, escribir código no se siente lento incluso en archivos grandes.

